Lidando com sensores em dispositivos móveis

Documentação do Projeto -
Wear OS API 30+ | Ultima Interação 20/04/2026

Este documento explica o funcionamento do aplicativo **AppDoma proposto em uma avaliação pratica da Estácio**, desenvolvido para smartwatches com **Wear OS**. O app detecta saídas de áudio disponíveis no relógio (alto-falante ou fone Bluetooth), se comunica com o usuário por voz usando **Text-to-Speech (TTS)** e facilita a conexão de fones Bluetooth.

1. O que o app faz

O AppDoma foi criado para auxiliar funcionários com necessidades especiais, fornecendo instruções e alertas sonoros diretamente no smartwatch. Ao abrir o app, ele verifica automaticamente qual saída de áudio está disponível e exibe o status na tela. O usuário pode pressionar o botão **Falar** para ouvir uma mensagem de assistência, ou o botão **Conectar Bluetooth** para parear um fone.

Funcionalidades principais:

- **Detecção de áudio:** verifica se o relógio tem alto-falante ou fone Bluetooth conectado.
- **Text-to-Speech:** converte texto em fala em português do Brasil.
- **Callback dinâmico:** detecta automaticamente quando um fone BT é conectado ou desconectado.
- **Atalho Bluetooth:** abre as configurações BT do sistema com um botão.

---

2. Estrutura do projeto

O projeto é dividido em 4 arquivos Java, cada um com uma responsabilidade clara. Essa separação facilita a manutenção — se precisar mudar algo de Bluetooth, por exemplo, você mexe apenas no BluetoothHelper.

```
AppDoma/
■■■ MainActivity.java → Tela principal. Liga tudo.
■■■ AudioHelper.java → Detecta alto-falante e fone BT.
■■■ TtsHelper.java → Converte texto em fala (TTS).
■■■ BluetoothHelper.java → Abre as configurações Bluetooth.
```

3. AudioHelper.java

Esta classe é responsável por verificar quais saídas de áudio estão disponíveis no dispositivo. Ela usa o **AudioManager** do Android para consultar os dispositivos conectados e registrar um **callback** que avisa o app quando um fone Bluetooth é conectado ou desconectado.

Como funciona:

- **audioOutputAvailable(type):** percorre a lista de dispositivos de saída e retorna true se o tipo pedido (speaker ou BT) estiver presente.
- **isSpeakerAvailable():** atalho que chama o método acima com o tipo TYPE_BUILTIN_SPEAKER.
- **isBluetoothHeadsetConnected():** atalho para TYPE_BLUETOOTH_A2DP (protocolo de áudio BT).
- **registerAudioDeviceCallback():** registra dois blocos de código — um executado quando BT conecta, outro quando desconecta.

```
// Verifica se um tipo de saída de áudio está disponível
public boolean audioOutputAvailable(int type) {
// Se o dispositivo nem tem saída de áudio, retorna falso
if (!context.getPackageManager()
.hasSystemFeature(PackageManager.FEATURE_AUDIO_OUTPUT)) {
return false;
}
// Percorre todos os dispositivos de saída
AudioDeviceInfo[] devices =
audioManager.getDevices(AudioManager.GET_DEVICES_OUTPUTS);
for (AudioDeviceInfo device : devices) {
if (device.getType() == type) return true; // Encontrou!
}
return false; // Não encontrou
}

// Registra callbacks chamados automaticamente quando BT muda
audioManager.registerAudioDeviceCallback(new AudioDeviceCallback() {
@Override
public void onAudioDevicesAdded(AudioDeviceInfo[] addedDevices) {
if (isBluetoothHeadsetConnected()) onBluetoothConnected.run();
}
@Override
public void onAudioDevicesRemoved(AudioDeviceInfo[] removedDevices) {
if (!isBluetoothHeadsetConnected()) onBluetoothDisconnected.run();
}
}, null);
```

4. TtsHelper.java

Esta classe gerencia o **Text-to-Speech** — o motor de voz do Android que transforma texto em fala. A inicialização do TTS é **assíncrona**, ou seja, leva alguns milissegundos para ficar pronto. Por isso, se o botão for clicado muito rápido, o texto é salvo em uma fila (**pendingText**) e falado assim que o motor estiver pronto.

Pontos importantes:

- **Inicialização assíncrona:** o TTS não está pronto no instante em que é criado. O método onInit() avisa quando está.
- **pendingText:** guarda o texto caso speak() seja chamado antes do TTS inicializar.
- **Fallback para inglês:** emuladores frequentemente não têm o pacote de voz pt-BR instalado. Se isso acontecer, o app usa inglês automaticamente.
- **shutdown():** libera a memória do TTS quando o app fecha — evita vazamento de recursos.

```
@Override
public void onInit(int status) {
if (status == TextToSpeech.SUCCESS) {
int result = tts.setLanguage(new Locale("pt", "BR"));

// Se pt-BR não estiver disponível, usa inglês
if (result == TextToSpeech.LANG_MISSING_DATA ||
result == TextToSpeech.LANG_NOT_SUPPORTED) {
tts.setLanguage(Locale.ENGLISH);
}

isReady = true;

// Havia texto esperando? Fala agora.
if (pendingText != null) {
speak(pendingText);
pendingText = null;
}
}
}

public void speak(String text) {
if (isReady) {
tts.speak(text, TextToSpeech.QUEUE_FLUSH, null, null);
} else {
pendingText = text; // Guarda para falar quando pronto
}
}
```

5. BluetoothHelper.java

Classe simples com um único método estático que abre a tela de configurações Bluetooth do Wear OS. Em vez de mostrar uma mensagem de erro quando nenhum fone está conectado, o app leva o usuário diretamente para onde ele pode resolver o problema.

Por que método estático?

Como a classe não tem estado (não guarda nenhuma variável), não faz sentido instanciá-la. Com **static**, basta chamar diretamente: `BluetoothHelper.openBluetoothSettings(this)`.

```
public static void openBluetoothSettings(Context context) {
Intent intent = new Intent(Settings.ACTION_BLUETOOTH_SETTINGS);

// Abre em nova tela e limpa a navegação anterior
intent.addFlags(Intent.FLAG_ACTIVITY_NEW_TASK |
Intent.FLAG_ACTIVITY_CLEAR_TASK);

// Extras específicos do Wear OS:
intent.putExtra("EXTRA_CONNECTION_ONLY", true); // Só mostra conexão
intent.putExtra("EXTRA_CLOSE_ON_CONNECT", true); // Fecha ao conectar
intent.putExtra(
"android.bluetooth.devicepicker.extra.FILTER_TYPE", 1); // Só áudio

context.startActivity(intent);
}
```

6. MainActivity.java

É a tela principal do app e o ponto central que conecta todos os outros arquivos. Ela inicializa os helpers, associa os botões às ações e garante que a UI seja atualizada corretamente quando algo mudar.

Detalhe importante — runOnUiThread:

Os callbacks do AudioHelper chegam em uma **thread de fundo** (background thread). Modificar a UI a partir de uma thread de fundo causa erro no Android. Por isso usamos **runOnUiThread()** para garantir que a atualização do texto aconteça na thread principal.

```
audioHelper.registerAudioDeviceCallback(
// Executado quando fone BT conecta:
() -> runOnUiThread(() -> {
tvStatus.setText("Bluetooth conectado"); // Atualiza UI
ttsHelper.speak("Fone conectado."); // Fala
}),
// Executado quando fone BT desconecta:
() -> runOnUiThread(() -> {
tvStatus.setText("Bluetooth desconectado");
ttsHelper.speak("Fone desconectado.");
})
);

// Botão Falar
btnSpeak.setOnClickListener(v ->
ttsHelper.speak("Assistência ativa para funcionários.")
);

// Botão Bluetooth
btnBluetooth.setOnClickListener(v ->
BluetoothHelper.openBluetoothSettings(this)
);

// Libera o TTS quando o app fecha
@Override
protected void onDestroy() {
super.onDestroy();
ttsHelper.shutdown();
}
```

7. AndroidManifest.xml e Layout

AndroidManifest.xml

Todo app Android precisa declarar no Manifest quais permissões usa e quais telas tem. Sem declarar as permissões de Bluetooth, o sistema bloqueia o acesso mesmo que o código esteja correto.

```
<!-- Permissões de Bluetooth obrigatórias -->
<uses-permission android:name="android.permission.BLUETOOTH" />
<uses-permission android:name="android.permission.BLUETOOTH_CONNECT" />
<uses-permission android:name="android.permission.BLUETOOTH_SCAN" />

<!-- Informa que é um app Wear OS -->
<uses-feature android:name="android.hardware.type.watch" />

<!-- App funciona sem celular pareado -->
<meta-data android:name="com.google.android.wearable.standalone"
android:value="true" />
```

activity_main.xml — Layout Wear OS

O **BoxInsetLayout** é o layout recomendado para Wear OS pois protege o conteúdo de ser cortado nas bordas arredondadas da tela do relógio. Dentro dele, um **LinearLayout vertical** empilha os componentes centralizados.

```
<androidx.wear.widget.BoxInsetLayout> <!-- Protege bordas redondas -->
<LinearLayout
android:orientation="vertical"
android:gravity="center">

<TextView android:id="@+id/tvStatus" /> <!-- Status do áudio -->
<Button android:id="@+id/btnSpeak" /> <!-- Botão Falar -->
<Button android:id="@+id/btnBluetooth" /><!-- Botão Bluetooth -->

</LinearLayout>
</androidx.wear.widget.BoxInsetLayout>
```

8. Problemas encontrados e soluções

Botão Falar não funcionava

Causa: O TTS tem inicialização assíncrona. Quando o botão era clicado logo após abrir o app, isReady ainda era false e o speak() não executava nada.

Solução: Criei o campo pendingText que guarda o texto e o fala assim que o onInit() confirmar que o TTS está pronto.

Sem áudio no emulador

Causa: Emuladores Wear OS frequentemente não têm o pacote de voz pt-BR instalado, o que fazia o TTS falhar silenciosamente.

Solução: Adicionei um fallback para Locale.ENGLISH quando pt-BR não está disponível.

Sem áudio no emulador

Causa: Emulador sem TTS Engine

Solução: Não Consegui nenhuma solução em código por se tratar de um problema com o emulador (Pode ser instalado via ADB ou Playstore).